


```
import numpy as np
from scipy.optimize import linprog
```

```
def solve_ot_lp(a, b, C, order='C'):
```

```
    """
```

```
    Solve discrete Kantorovich OT:
```

```
     $\min_{\{\pi \geq 0\}} \langle C, \pi \rangle$  s.t.  $\pi \mathbf{1} = a, \pi^T \mathbf{1} = b$ 
```

```
    Returns  $(\pi^*, \text{cost})$ .
```

```
    Vec convention:  $x = \text{vec}(\pi)$  with row-stacking if order='C'.
```

```
    """
```

```
    n, m = len(a), len(b)
```

```
    assert C.shape == (n, m), "C must be nxm matching a,b"
```

```
    c = C.reshape(-1, order=order)
```

```
    A_eq = np.vstack([np.kron(np.eye(n), np.ones((1, m))),
                      np.kron(np.ones((1, n)), np.eye(m))])
```

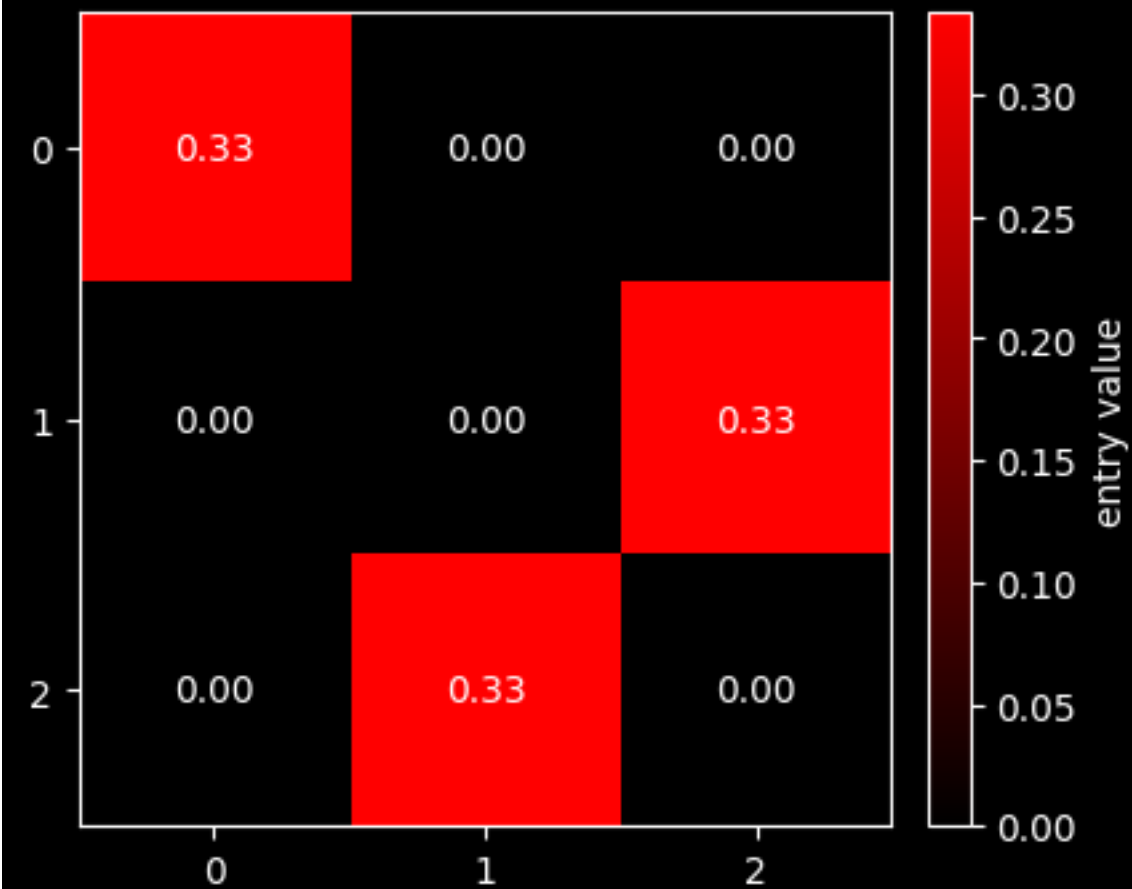
```
    b_eq = np.r_[a, b]
```

```
    res = linprog(c, A_eq=A_eq, b_eq=b_eq,
                  bounds=[(0, None)]*(n*m), method="highs")
```

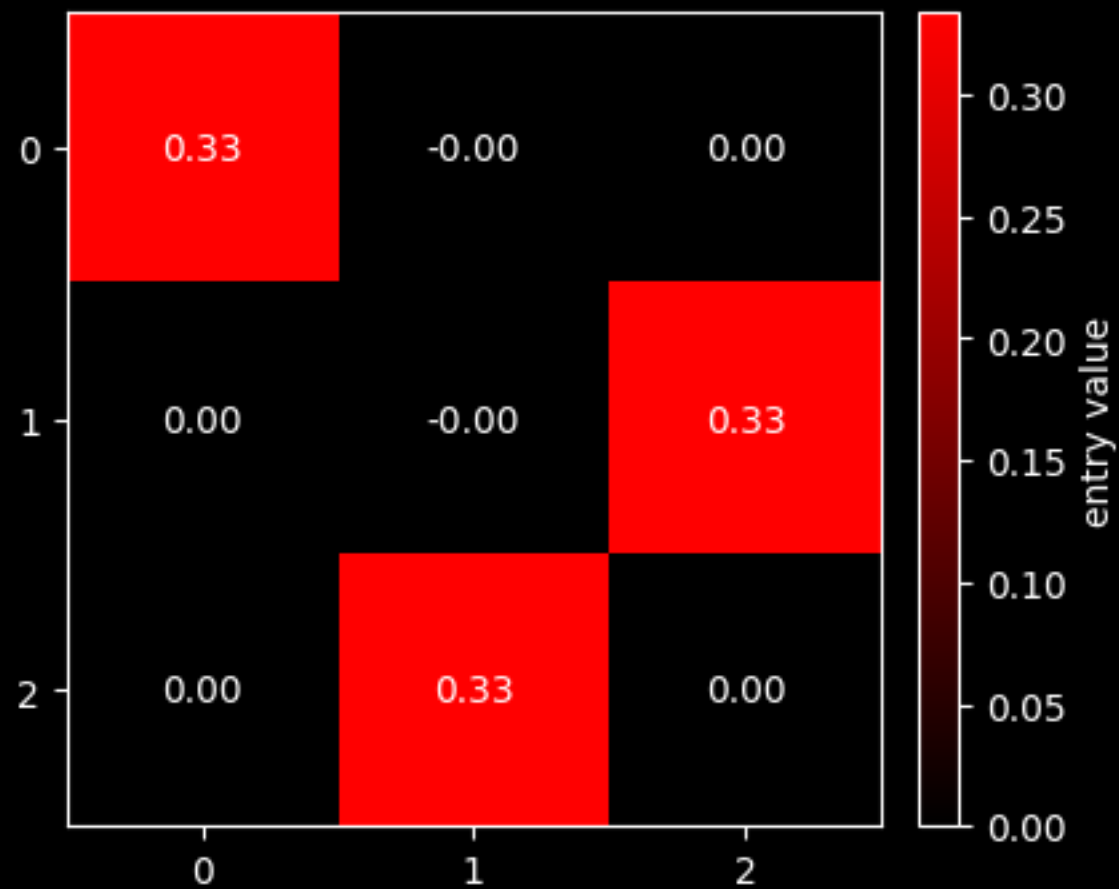
```
    pi = res.x.reshape((n, m), order=order)
```

```
    return pi, float((pi * C).sum())
```

Monge (Permutation)
Cost=1.500



Kantorovich (Coupling)
Cost=1.500



Interactive Demo: Kantorovich

```
import numpy as np
from scipy.optimize import linprog
```

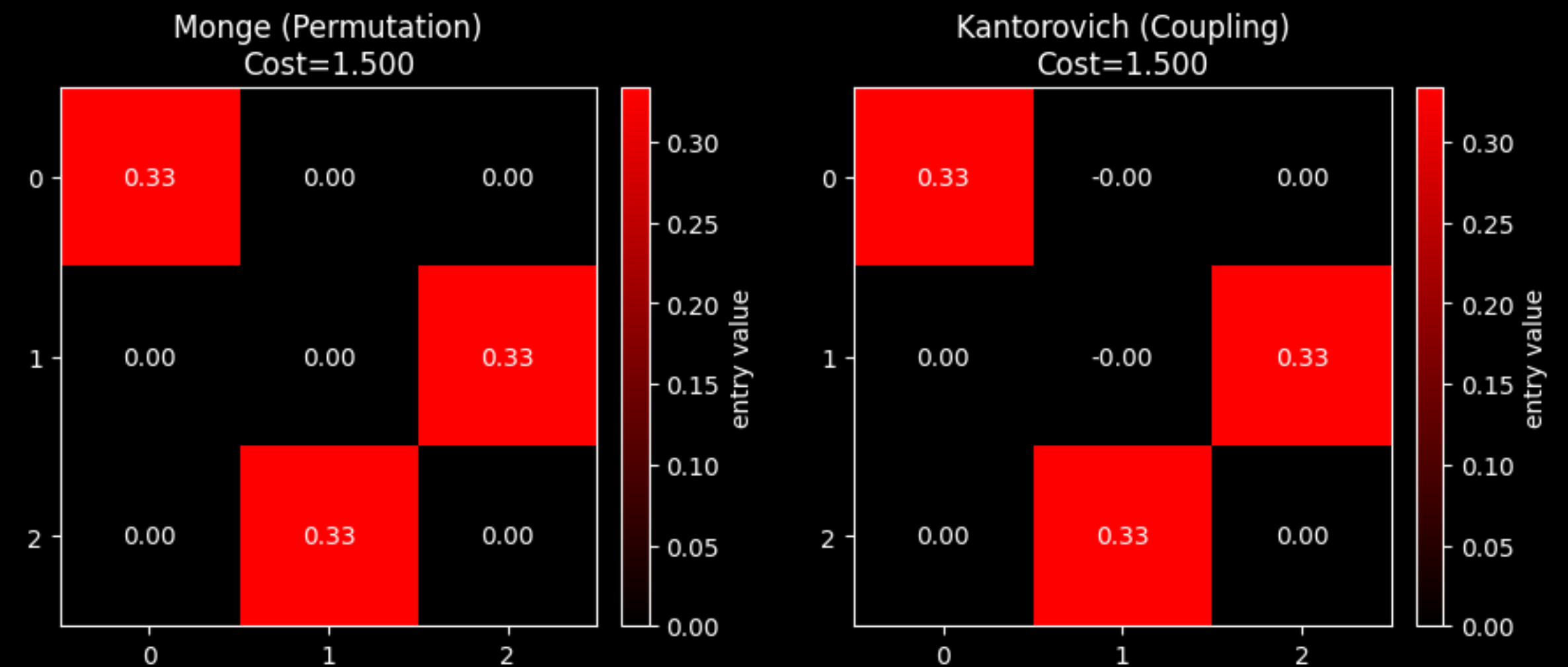
```
def solve_ot_lp(a, b, C, order='C'):
    """
    Solve discrete Kantorovich OT:
     $\min_{\pi \geq 0} \langle C, \pi \rangle$  s.t.  $\pi \mathbf{1} = a, \pi^T \mathbf{1} = b$ 
    Returns  $(\pi^*, \text{cost})$ .
    Vec convention:  $x = \text{vec}(\pi)$  with row-stacking if order='C'.
    """
```

```
    n, m = len(a), len(b)
    assert C.shape == (n, m), "C must be n×m matching a,b"
```

```
    c = C.reshape(-1, order=order)
    A_eq = np.vstack([np.kron(np.eye(n), np.ones((1, m))),
                      np.kron(np.ones((1, n)), np.eye(m))])
    b_eq = np.r_[a, b]
```

```
    res = linprog(c, A_eq=A_eq, b_eq=b_eq,
                  bounds=[(0, None)]*(n*m), method="highs")
```

```
    pi = res.x.reshape((n, m), order=order)
    return pi, float((pi * C).sum())
```



Summary & Teaser

- Key concepts learned:
 - Couplings, bistochastic matrices, Birkhoff, Kantorovich.
- Teaser: This formulation gives us one more thing